

Comparative Analysis of Firmware Integrity Assurance Methods: A Survey

S. Yashitha

yashitha.s2022@vitstudent.ac.in

K. Hailey

hailey.k2022@vitstudent.ac.in

G. Pavithra

pavithra.g2022@vitstudent.ac.in

R. Charanya

charanya.r@vit.ac.in

Abstract

With the increasing appearance of the Internet of Things (IoT) equipment, now accounting for approximately one third of all connected devices, it has become a significant concern. These devices often introduce weaknesses that make organizations easy targets for cyber hazards. The most important in this domain is yet one of the unseen areas, firmware security. Regular firmware analysis is necessary to patch the bug, fix weaknesses, and integrate new protective features. However, ensuring firmware integrity is far from simple. The variety of IOT architecture and the prevalence of ownership (closed source) firmware complicates both analysis and imitation. These factors make it difficult for researchers and developers to inspect firmware behaviour in a controlled environment. Over the years, many innovative ways have come up to overcome these challenges. This task presents a comprehensive review of solutions that are classified by various analysis approaches. It analyses existing obstacles in firmware security, evaluates the effectiveness of existing methods, and explores the areas where improvements are still necessary. The study acts as a practical guide for cyber security researchers, developers, and engineers, which aims to strengthen the IOT device safety through stronger firmware analysis and simulation strategies.

Key Words

Firmware Security; IoT Device Integrity; Cryptographic Hashing (SHA-256); Embedded System Security; Tamper Detection

Introduction

In today's digital world, we use a lot of IoT devices every day for various purposes, it brings us comfort and convenience [1]-[5]. All these devices are programmed with a code that gives them the ability to perform its intended function, called as firmware [6]. Security analysis is essential for both software and hardware [7]-[11]. To ensure device security, we verify firmware of IoT devices and check for vulnerabilities [12]. Unlike application-level software attacks, firmware-level attacks are harder

to detect. According to a 2021 Microsoft Security Signals report, more than 80% of enterprises experienced at least one firmware-related attack in the past two years. But only 29% of security budgets are allocated to protecting firmware, despite its critical role in system integrity (Microsoft). To analyse the security, most IoT devices need access to firmware. It can be analysed through many ways such as manual analysis, binary lifting, emulation [13]-[17]. As the IoT devices communicate through network, it can be another vulnerability. Firmware plays an important role in IoT devices, as it directly controls hardware. Hence it becomes a target for malicious uses. This unauthorized access or modification or replacement of firmware is known as Firmware tampering. It poses significant threat to the device as the attacker can gain control over the device by escaping traditional software security mechanisms. Once firmware is compromised, even after reformatting or reinstalling the OS, the attackers can gain access to the device. This paper explores a hardware-based firmware tamper detection system that uses hashing to assure firmware integrity. The main idea is to generate the golden hash (hash of an original and trusted system) and secure its storage. Later, the hash of the target device is calculated and compared with golden hash. Any mismatch in hash values can be treated as a tampering event. The system utilizes SHA256 to calculate the golden hash and target device hash value. The system is developed to be a simple, light weight, low cost and able to function offline with no cloud or internet dependencies. To enhance the reporting mechanism, both SD card logging and Bluetooth based alerting were adopted. This offers long term support and improved reliability. It also overcomes the limitations of solely depending on buzzers or LEDs. The system can also detect physical tamper using wire loop sensors to monitor physical access.

- SD Card Logging logs tamper incidents along with timestamp on an SD card, which is later used for evidence, analysis and traceability.
- Bluetooth-Based alerting is used to notify real-time tamper alerts to a paired mobile device or PC, thereby supporting remote,

wireless notifications without requiring internet access.

The system is developed with hardware components such as Arduino (ESP32), SD card module, micro-SD card and Bluetooth module. Compared to existing works that often depend on external authentication services or complex software stacks and costly, our approach emphasizes simplicity, cost-effectiveness, and ease of demonstration, making it highly suitable for academic use, prototyping, and security research. This approach offers a foundational step toward firmware trust verification for embedded systems, especially in environments where TPMs or Secure Boot are unavailable.

Motivation

The protection of IoT systems is increasing in prominence as IoT devices become integrated into homes, industries, hospitals, transportation and military. Security in IoT devices is often lacking and, in many cases, non-existent, particularly at the firmware level, which serves as the software that manages hardware, the logic of execution, and the communication protocols. Once this firmware is hacked, inserted with backdoors, inserted with malicious software or replaced entirely, the behavior of the entire logic will change. One of the biggest downfalls with firmware-based attacks is the ability to be undetected. After the firmware is hacked, the firmware often continues to function as though it was normal, while transmitting data and spying on further or changing the behavior of the device without any user consent or knowledge. This project is motivated by the fact that a lightweight, accessible, and reliable framework is needed to detect firmware modifying, particularly in devices that are unable to support complex secure boot mechanisms or for those that exist offline, as well as in remote areas. Within academic papers and industry reports, firmware attacks have been responsible for recently reported breaches comparing the VPNFilter, and MoonBounce malware that infected routers and embedded devices by manipulating the firmware. These attacks illustrate the security implications, that insecure firmware can provide as a backdoor for subsequent payloads to be launched once the firmware is altered.

Objective

The intent of this project is to develop a firmware verification system using a low-cost embedded

device that can read firmware memory from an IoT device (such as an Arduino or NodeMCU), compute a secure hash (e.g., SHA-256) of the binary code, store a trusted hash at some time, during startup, as an 'original' or 'golden' hash for that binary, check and compare any hashes that become created against the golden version, while notifying (buzzer, red LED, OLED display) and logging the event when hashes do not match, and logging timestamps of verified hashes for audit requirements. The core design of the system will focus on being offline - an advantage for devices that operate undercover, in space like agricultural fields, rural south pacific health systems, or military sensors. The solution is being designed around an ESP32 microcontroller, since it has Wi-Fi capabilities, SPI/UART interface, and has cryptographic libraries. It is well suited to embedded security applications.

Literature Survey

Today's firmware validation techniques tend to fall into one of three categories that each have their drawbacks:

1. Secure Boot Systems: They are usually used in commercial or enterprise IoT platforms. They work by verifying cryptographic signatures of the firmware before booting. But secure boots need special hardware, fuse-burning, or embedded keys. However, they are inconvenient for DIY efforts and low-cost deployments [18].
2. Cloud-Based Firmware Monitoring: Some platforms monitor firmware updates through cloud dashboards or backend analytics. Although it is very useful in centralized management, they completely depend on network availability and can be attacked if the attacker shuts down communication or conducts the attack offline [19].
3. Behavioural Anomaly Detection: Some techniques try to find abnormal behavior on devices—such as unusual use of resources or unusual patterns of communication. These can lead to false positives and generally do not identify if the firmware itself is changed [20][21].

S.NO	Paper	Author(s)	Methodology	Advantages	Disadvantages
1	MoonBounce: UEFI firmware-level APT framework	Kaspersky (2022)	Self-published works on firmware memory forensics and UEFI malware detection illuminate bootloader step threats.[22]	These works point out advanced persistent threats (APTs) and offer insights into the sophistication of persistently firmware-level threats.	Focuses only on UEFI; limited hardware diversity in tests.
2	RAM-Based Firmware Attestation for IoT Security: A Representation Learning Framework	Asif Iqbal, Usman Zia, Muhammad Naveed Aman, Biplab Sikdar	RAM trace-based attestation [23]	Avoids the overhead of storing or retrieving the authentic firmware binary	Dependable collection of RAM traces is needed, may prove difficult for some hardware or settings
3	Framework to analyze and exploit the smart-home IoT firmware	R. P. Singh, S. N. Dwivedi, and P. P. Bhattacharya	Performs entropy and compression profiling.[24]	Useful as an initial filter for large firmware images where only portions may be altered, and can discover both recognized and unique zero-day exploits.	Encrypted or compressed but valid firmware portions may create inaccurate results.
4	Advanced firmware validation via heterogeneous data fusion (PLOS ONE)	Peng Xiao, Linjiang Xie, Feilu Hang, Hanruo Li	Fuse static, dynamic, telemetry, metadata signals into anomaly detection [25]	Broad visibility; multi-signal robustness	Complex; heavy engineering; requires diverse data pipelines
5	Eclypsium / industry reports (Gigabyte backdoor, UEFI compromises)	Eclypsium / industry report authors	Forensic-update analysis, signature/behavior checks [26]	Real-world, high relevance; immediately actionable	Vendor-dependent; may lack peer-review; requires rapid vendor patching
6	Tamper Detection in Industrial Sensors (PMC article)	William Villegas-Ch, Jaime Govea, Angel Jaramillo-Alcazar	Sensor-output anomaly detection for firmware tamper [27]	Validated in industrial context; domain-tailored	Environment-specific thresholds; prone to false positives
7	ChkUp / Firmware update vulnerability frameworks (“Your Firmware Has Arrived”)	Yuhao Wu, Jinwen Wang, Yujie Wang, Shixuan Zhai, Zihan Li, Yi He, Kun Sun, Qi Li, Ning Zhang	Emulated dynamic testing of update mechanisms, cross-language control flow analysis [28]	Finds practical flaws pre-deployment	Requires manual effort; emulation may diverge from real hardware
8	Runtime kernel/firmware integrity checks without extra IP (SAE)	Shyju Thekkumbadan, Sreedhar Reddy Pacharla, Jipin Jose	Periodic memory/code integrity checks via ARM TrustZone [29]	Continuous monitoring; no extra hardware needed	Performance overhead; windows of vulnerability between checks

9	Utilization of Impedance Disparity from Switching Activities to Monitor Firmware Activities	Md Sadik Awal, Christopher Thompson, Md Tauhidur Rahman	Uses ML algorithms to differentiate by observing the changes in behavior [30]	Verifies side channels without intrusive hardware, light weight monitoring	Sensitive to hardware variations; requires classifiers tailored per device.
10	Swarm-Net: Firmware Attestation in IoT Swarms using Graph Neural Networks and Volatile Memory (2024)	V. Kohli et al.	SRAM-based graph neural networks for swarm-scale firmware attestation [31]	99.96% - Works at scale for IoT swarms	Training overhead may not be directly applicable to all hardware systems
11	PACCOR4ESP : Embedded Device Security Attestation using Platform Attribute Certificates (2024)	T. Grübl et al.	Adds PACCOR certificates for ESP32, including firmware hash, bootloader hash, and GPIO config.[32]	Supports RAS, CRAFT, and SEDA frameworks, and acquires several device characteristics	Implementation for ESP32, Overhead in certificate handling.
12	A Review of IoT Firmware Vulnerabilities and Auditing Techniques (2024)	O. Akinyele et al.	Review papers on firmware auditing (interface, RE, blockchain, ML, Trust Zone, SMART) [33]	Global view of techniques, Gaps are identified in the state-of-the-art.	Review only, no new experimental validation
13	A Security Framework for Increasing Data and Device Integrity in Internet of Things Systems (2023)	J. P. B. da Silva et al.	Offers framework comparison for attestation (IoTAttest, DIAT, CRAFT, US-AID).[34]	Review focus: scalability, trust anchors, hardware attestation	No empirical evaluation of the prototype.
14	A Highly Secure IoT Firmware Update Mechanism Using Blockchain (2022)	M. El-Hajj, A. F. M. Sultan, H. Otok, R. Mizouni	Blockchain + IETF SUIT is utilized for firmware updates security.[35]	Blocks malicious update injection, ensuring logs are tamper-proof.	Blockchain increases latency and storage overhead. Heavy resource constraints.
15	Future-Proofing IoT with Zero Trust Security: Trends and Challenges (2024)	P. Sharma et al.	Zero Trust application on IoT firmware integrity (trust scoring via AI, TPM attestation).[36]	Emphasized quantum-resilient trends, integrated with firmware integrity checks.	Conceptual work with a few implementations so far.

16	Firmware Watermarking Techniques for Tamper Detection (2025)	A. Mehta, R. Singh, P. Bansal	Detects unauthorized changes by embedding watermarks in firmware binaries.[37]	Modification detection, Constraints of IoT devices	Stripping by sophisticated attackers needs compiler integration.
17	Lightweight Remote Attestation Protocols for IoT Devices (2025)	R. Kumar, S. Das, A. Verma	Lightweight attestation with no heavy crypto designed for low-cost IoT [38]	Works on resource-constrained devices, with low overhead	Simple device models only, vulnerable to advanced runtime attacks.
18	IMUP: Integrity-Centric Modular Update Platform (2025)	S. Park, Y. Lee, H. Kim	Uses chameleon hashing for firmware update integrity on a per-module basis.[39]	Safeguards against modular/custom firmware update and ensures high integrity verification.	Complex hashing management; multi-module systems incur a cost due to computational overhead.
19	Aye: A Trusted Forensic Method for Firmware Tampering Attacks (2023)	Yipeng Zhang, Ye Li, Zhoujun Li	Supports the JTAG interface to capture firmware memory from a PLC (Programmable Logic Controller) and compares it with a trusted ones to detect tampering.[40]	Highly accurate forensic detection; compatible with existing industrial hardware (PLC) without firmware modification.	Requires physical access and a JTAG port; cannot prevent tampering in real time; limited to devices with exposed debugging interfaces.
20	Firmware Attestation in IoT Swarms Using Relational Graph Neural Networks (2025)	Abdelkabar Rouagubi, Chaymae El Youssofi, Khalid Choug dali	Models an IoT swarm as a graph and uses relational graph neural networks (RGNNs) on runtime SRAM snapshots to detect anomalous firmware behavior without needing firmware binaries.[41]	Extremely accurate attestation (>99.9%); resilient to noise, replay, and missing data; scalable across distributed swarms.	Requires coordinated devices, labeled training data, and synchronicity and not suitable for standalone devices.
21	Uncovering EDK2 Firmware Flaws: Insights from Code Audit Tools (2024)	Mahsa Farahani, Ghazal Shenavar, Ali Hosseinghorban, Alireza Ejlali	Uses code audit tools to EDK2 UEFI firmware to identify security weaknesses.[42]	Supports audit tools for firmware; applicable to UEFI implementations.	Tools are not firmware-specific, also effectiveness depends on tool flexibility.
22	Exploring Firmware-Based Anti-forensics in IoT	Mariya Shafat Kirmani, M. Tariq Banday	Explores anti-forensic methods on BeagleBone Black to hide	Practical demonstration of firmware	Focused on a single board; needs more

	Devices: Techniques and Implications (2024)		tampering and suggests mitigations.[43]	tampering, suggests actions.	broader generalization.
23	Balancing Accuracy and Efficiency in Vehicular Network Firmware Vulnerability Detection (FirmPB Framework) (2025)	Yibo Wang (et al.)	Uses deep-learning-based framework (FirmPB) for efficient firmware vulnerability detection in vehicular networks.[44]	High accuracy in detection, and it is faster than traditional methods.	Heavily dependent on deep learning models.
24	UEFI Vulnerability Signature Generation using Static and Symbolic Analysis (STASE) (2024)	Md Shafiuzzaman, Achintya Desai, Laboni Sarker, Tevfik Bultan	Combines static and symbolic analysis of UEFI firmware to produce vulnerability signatures. [45]	Detects known and unknown faults, can be used for complex firmware.	Dependent on LLVM bitcode and rule configuration, symbolic execution needs heavy resources.
25	Securing Firmware Updates: Addressing Security Challenges in UEFI Capsule Update Mechanisms (2024)	Younus A Shaik, Pankaj Yadav	Analyzes vulnerabilities in UEFI capsule update mechanisms and suggests mitigations techniques.[46]	Focuses directly on the attack layer for firmware tampering.	Concentrates on Analytical methods than experimental methods and lacks implementation support.

The literature survey focuses on the remarkable progress toward addressing firmware security in the context of IoT ecosystems. Various approaches including secure boot mechanisms, cloud-attestation, blockchain, anomaly detection, machine learning-based monitoring, and hardware-assisted integrity checks have been thoroughly studied. While these papers reflect the effectiveness of sophisticated cryptographic, forensic, and behavioral mechanisms, they also indicate important limitations when deployed in real, low-cost IoT deployments. For example, secure boot and trusted execution environments such as ARM TrustZone provide strong integrity guarantees but require special hardware support and fuse-based key burning, which are impossible for low-cost or community/ maker-developed IoT devices. Cloud-based monitoring capabilities facilitate centralized management but are extremely dependent on ongoing network connectivity, subjecting them to offline attacks and inappropriate for devices to be deployed in distant or isolated locations. Similarly,

blockchain-protected update mechanisms and ML/AI-powered anomaly detection of device activity guarantee new methods but introduce substantial computational overhead, latency, and storage requirements outside of the majority of lightweight IoT solutions. Moreover, the anomaly detection methods are prone to false positives as they are likely to confuse regular fluctuations in device operation as malicious interference. Vendor-specific approaches such as UEFI firmware audits or manufacturer-initiated forensic reporting also limit the applicability of such methods to ubiquitous deployment in heterogeneous IoT environments. Collectively, the survey reveals that while research into firmware integrity has come a long way, opening remains for an inexpensive, real-time, and offline-friendly simple method of firmware verification that can be practiced in a wide range of IoT devices without the requirement for proprietary networks or extensive resources.

GAPS IDENTIFIED

Based on the above discussion, several persistent gaps can be recognized within existing practices. First, there is overdependence on special hardware such as TPMs, Secure Boot modules, or vendor-based platforms, further inhibiting adoption in low-cost embedded systems. Second, most cloud-based and blockchain-based platforms are based on sustained internet connectivity and are hence unsuitable for offline or resource-constrained environments. Third, data fusion, machine learning, or graph-based attestation methods, while being robust, demand a lot of computational resources and introduce additional latency, which is not compatible with the light nature of IoT devices. Fourth, most detection methods work retrospectively by identifying tampering only after the compromise through forensic or memory analysis and not provide proactive real-time detection. Fifth, behavior-based anomaly detection suffers from reliability issues since false positives for regular device oscillations are classified as attacks. Finally, most contributions are vendor specific, firmware specific, or architecture specific and therefore are not scalable across the heterogeneous IoT world. All these gaps lead to the need for a low-cost, vendor-independent, and real-time firmware assurance of integrity system that will operate without heavy infrastructure or continuous reliance on the internet.

Conclusion

Ensuring the integrity of firmware is an important aspect of the creation of safe and reliable IOT ecosystems. The proposed project presents a direct effective solution yet to detecting unauthorized firmware modifications using hardware-based cryptographic approaches. By taking advantage of components like ESP 32 microcontroller, SD card storage, buzzer and OLED displays combined with SHA-256 hashing, the system provides real-time alert whenever the system is detected in firmware integrity. The benefit of this project is its simplicity and accessibility. It does not depend on the cloud infrastructure, making it suitable to work offline or resource-wide environments. Its cost-effective and modular designs also make the safety features to integrate in embedded equipment, which enables the use of safety features in embedded devices. Beyond its technical properties, the project contributes a fundamental layer of defence to the growing scenario of IOT weaknesses. Firmware by priority to integrity - frequent unseen aspects in device fines - This work gives groundwork for further innovations

in embedded device safety. The architecture not only acts as a proof of the concept but also as a scalable framework, on which more advanced security can be constructed.

Future work

Digital signature verification: Future recurrences of ESP 32 firmware must use public-key cryptography (e.g. RSA or ECC) to improve safety beyond simple hashing. It effectively prevents unwanted re-hashing and ensures increased validity using a private key to signing firmware and a related public key to verify it on ESP 32. Safe over-the-air (OTA) update: It would be a significant improvement to allow safe OTA update capacity as the IOT device is more remote and disarmed. This will enable verified firmware updates obtained by equipment without the need for physical access, thus preserving system integrity during version changes. Remote logging and alert: AWS IOT, Azure, or Firebase can be made possible by integrating cloud services such as firmware. The hash may increase mismatched, push notification, or email visibility and response through a notification or SMS alert of real-time logging and tampering efforts. ML behavioural Anomaly Detection: By integrating the light machine learning model directly on the microcontroller, the device may be able to detect and report strange runtime behaviour. This additional layer of detection might find more advanced attacks that get Around Hash Checks. Reusable firmware security modules: By packaging the hash and verification process as a reusable library or module, developers can easily add firmware verification to other microcontroller architecture. This may promote the use of firmware security proceeds among the maker and open-source communities.

References

- [1] P. Gandhi, S. Khanna, and S. Ramaswamy, "Which industries are the most digital (and why)?" [Online]. Available: <https://hbr.org/2016/04/a-chart-that-shows-which-industries-are-the-most-digital-and-why>, Accessed on: Apr. 1, 2016.
- [2] K. Wiles, "First all-digital nuclear reactor system in the U.S. installed at Purdue University," [Online]. Available: <https://www.purdue.edu/newsroom/releases/2019/Q3/first-all-digital-nuclear-reactor-controlsystem-in-the-u.s.-installed-at-purdue-university.html>, Accessed on: Jul. 8, 2019.
- [3] O. Friha, M. A. Ferrag, L. Shu, L. Maglaras, and X. C. Wang, "Internet of things for the future of smart agriculture: A comprehensive survey of emerging technologies," *IEEE/CAA J. Autom. Sinica*, vol.8, no. 4, pp.718–752, Apr. 2021.

- [4] M. A. Ferrag, L. Shu, and K. K. R. Choo, "Fighting COVID-19 and future pandemics with the internet of things: Security and privacy perspectives," *IEEE/CAA J. Autom. Sinica*, vol. 8, no. 9, pp. 1477–1499, Sept. 2021.
- [5] StatInvestor, "Internet of things-number of connected devices worldwide 2015–2025," [Online]. Available: <https://statinvestor.com/ata/33967/iot-number-of-connected-devices-worldwide/>. Access on: Feb. 10, 2022.
- [6] Vasant Raval, "The Practical Aspect: Challenges of Security Log Management", in *ISACA Virtual Journal*, Nov. 2017. [Online]. Available: <https://uefi.org/sites/default/files/resources/Getting%20a%20Handle%20on%20Firmware%20Security%2011.11.17%20Final.pdf>
- [7] J. Zhang, L. Pan, Q.-L. Han, C. Chen, S. Wen, and Y. Xiang, "Deep learning based attack detection for cyber-physical system cybersecurity: A survey," *IEEE/CAA J. Autom. Sinica*, vol. 9, no. 3, pp. 377–391, Mar. 2022.
- [8] Y. Miao, C. Chen, L. Pan, Q.-L. Han, J. Zhang, and Y. Xiang, "Machine learning-based cyber attacks targeting on controlled information: A survey," *ACM Comput. Surv.*, vol. 54, no. 7, p. 139, Sept. 2022.
- [9] G. J. Lin, S. Wen, Q.-L. Han, J. Zhang, and Y. Xiang, "Software vulnerability detection using deep neural networks: A survey," *Proc. IEEE*, vol. 108, no. 10, pp. 1825–1848, Oct. 2020.
- [10] J. Y. Qiu, J. Zhang, W. Luo, L. Pan, S. Nepal, and Y. Xiang, "A survey of android malware detection with deep neural models," *ACM Comput. Surv.*, vol. 53, no. 6, p. 126, Nov. 2020.
- [11] N. Sun, J. Zhang, P. Rimba, S. Gao, L. Y. Zhang, and Y. Xiang, "Data-driven cybersecurity incident prediction: A survey," *IEEE Commun. Surv. Tutorials*, vol. 21, no. 2, pp. 1744–1772, Dec. 2018.
- [12] X. Feng, X. Zhu, Q. -L. Han, W. Zhou, S. Wen and Y. Xiang, "Detecting Vulnerability on IoT Device Firmware: A Survey," in *IEEE/CAA Journal of Automatica Sinica*, vol. 10, no. 1, pp. 25–41, January 2023, doi: 10.1109/JAS.2022.105860.
- [13] D. Davidson, B. Moench, S. Jha, and T. Ristenpart, "FIE on firmware: Finding vulnerabilities in embedded systems using symbolic execution," in *Proc. 22nd USENIX Conf. Security*, Washington, USA, 2013, pp. 463–478.
- [14] Y. David and E. Yahav, "Tracelet-based code search in executables," in *Proc. 35th ACM SIGPLAN Conf. Programming Language Design and Implementation*, New York, USA, 2014, pp. 349–360.
- [15] Y. Shoshitaishvili, R. Y. Wang, C. Hauser, C. Kruegel, and G. Vigna, "Firmallice-automatic detection of authentication bypass vulnerabilities in binary firmware," in *Proc. 22nd Annu. Network and Distributed System Security Symp.*, San Diego, USA, 2015.
- [16] D. D. Chen, M. Woo, D. Brumley, and M. Egele, "Towards automated dynamic analysis for Linux-based embedded firmware," in *Proc. 23rd Annu. Network and Distributed System Security Symp.*, San Diego, USA, 2016.
- [17] Z. M. Jiang, J. J. Bai, K. J. Lu, and S. M. Hu, "Fuzzing error handling code using context-sensitive software fault injection," in *Proc. 29th USENIX Conf. Security Symp.*, USENIX Association, 2020, pp. 146.
- [18] C. Profentzas, M. Günes, Y. Nikolakopoulos, O. Landsiedel and M. Almgren, "Performance of Secure Boot in Embedded Systems," 2019 15th International Conference on Distributed Computing in Sensor Systems (DCOSS), Santorini, Greece, 2019, pp. 198–204, doi: 10.1109/DCOSS.2019.00054. keywords: {Kernel;Security;Read only memory;Internet;Embedded systems; Hardware;Embedded Systems, Internet of Things, Secure Boot, System Security}
- [19] Ayan Chatterjee, Bestoun S. Ahmed, IoT anomaly detection methods and applications:A survey, *Internet of Things*, Volume 19, 2022, 100568, ISSN 2542-6605, <https://doi.org/10.1016/j.iot.2022.100568>.
- [20] Achiluzzi, E., Li, M., Georgy, M. F. A., & Kashef, R. (2022). Exploring the use of data-driven approaches for anomaly detection in the Internet of things (IoT) environment. *arXiv preprint arXiv:2301.00134*.
- [21] Palaniappan, K., Duraipandi, B., & Balasubramanian, U. M. (2024). Dynamic behavioral profiling for anomaly detection in software-defined IoT networks: A machine learning approach. *Peer-to-Peer Networking and Applications*, 17(4), 2450-2469.
- [22] Kaspersky Lab, "MoonBounce: UEFI Firmware-Level APT Framework," 2022. [Online]. Available: <https://securelist.com/moonbounce-the-dark-side-of-uefi-firmware/105468/>
- [23] A. Iqbal, U. Zia, M. N. Aman, and B. Sikdar, "RAM-Based Firmware Attestation for IoT Security: A Representation Learning Framework," *IEEE*, 2024.
- [24] R. P. Singh, S. N. Dwivedi, and P. P. Bhattacharya, "A Framework to Analyze and Exploit Smart-Home IoT Firmware," *Proc. IEEE Intl. Conf. on Computational Intelligence and Communication Technology (CICT)*, 2024.
- [25] P. Xiao, L. Xie, F. Hang, and H. Li, "Advanced technique for firmware security analysis through heterogeneous data fusion and knowledge mapping," *PLOS ONE*, vol. 20, no. 4, Apr. 11, 2025, Art. no. e0319660. doi: 10.1371/journal.pone.0319660
- [26] Eclysium, "Gigabyte Motherboard Firmware Backdoor Analysis," Eclysium Research Report, 2023.
- [27] F. M. Awan et al., "Tamper Detection in Industrial Sensors," *PMC*, 2023.

- [28] S. Oberoi et al., "A Deep Dive into Firmware Update Vulnerabilities and the Role of ChkUp," Medium Technical Report, 2023.
- [29] Z. Li et al., "Mechanism for Runtime Kernel Integrity Check without Additional IP," SAE Technical Paper, 2022.
- [30] M. S. Awal, C. Thompson, and M. T. Rahman, "Utilization of Impedance Disparity Incurred from Switching Activities to Monitor and Characterize Firmware Activities," *arXiv preprint*, Jan. 2023. [Online].
- [31] V. Kohli et al., "Swarm-Net: Firmware Attestation in IoT Swarms using Graph Neural Networks and Volatile Memory," *arXiv preprint arXiv:2408.05680*, 2024.
- [32] T. Grübl et al., "PACCOR4ESP: Embedded Device Security Attestation using Platform Attribute Certificates," *arXiv preprint arXiv:2407.14286*, 2024.
- [33] O. Akinyele et al., "A Review of IoT Firmware Vulnerabilities and Auditing Techniques," *Sensors*, vol. 24, no. 2, 2024.
- [34] J. P. B. da Silva et al., "A Security Framework for Increasing Data and Device Integrity in Internet of Things Systems," *Sensors*, vol. 23, no. 17, 2023.
- [35] M. El-Hajj, A. F. M. Sultan, H. Otok, and R. Mizouni, "A Highly Secure IoT Firmware Update Mechanism Using Blockchain," *Sensors*, vol. 22, no. 2, 2022.
- [36] P. Sharma et al., "Future-Proofing IoT with Zero Trust Security: Trends and Challenges," *ResearchGate preprint*, 2024.
- [37] A. Mehta, R. Singh, and P. Bansal, "Firmware Watermarking Techniques for Tamper Detection," *ResearchGate preprint*, 2025.
- [38] R. Kumar, S. Das, and A. Verma, "Lightweight Remote Attestation Protocols for IoT Devices," *ResearchGate preprint*, 2025.
- [39] S. Park, Y. Lee, and H. Kim, "IMUP: Integrity-Centric Modular Update Platform," *arXiv preprint arXiv:2507.19367*, 2025.
- [40] Y. Zhang, Y. Li, and Z. Li, "Aye: A Trusted Forensic Method for Firmware Tampering Attacks," *Symmetry*, vol. 15, no. 1, Art. 145, Jan. 2023, doi:10.3390/sym15010145.
- [41] A. Rouagubi, C. El Youssofi, and K. Chougali, "Firmware Attestation in IoT Swarms Using Relational Graph Neural Networks and Static Random Access Memory," *AI*, vol. 6, no. 7, Art. 161, Jul. 2025, doi:10.3390/ai6070161.
- [42] M. Farahani, G. Shenavar, A. Hosseinghorban, and A. Ejlali, "Uncovering EDK2 Firmware Flaws: Insights from Code Audit Tools," *arXiv preprint*, Sep. 2024. [Online].
- [43] M. S. Kirmani and M. T. Banday, "Exploring Firmware-Based Anti-forensics in IoT Devices: Techniques and Implications," *SN Computer Science*, vol. 5, Art. 1129, Dec. 2024, doi:10.1007/s42979-024-03476-y.
- [44] Y. Wang, "Balancing Accuracy and Efficiency in Vehicular Network Firmware Vulnerability Detection: A Fuzzy Matching Framework with Standardized Data Serialization," *Applied and Computational Engineering*, ACE Vol. 160, pp. 55-63, May 2025, doi:10.54254/2755-2721/2025.TJ23490.
- [45] M. Shafiuzzaman, A. Desai, L. Sarker, and T. Bultan, "UEFI Vulnerability Signature Generation using Static and Symbolic Analysis," *arXiv preprint*, Jul. 2024. [Online].
- [46] Y. A. Shaik and P. Yadav, "Securing Firmware Updates: Addressing Security Challenges in UEFI Capsule Update Mechanisms," *Int. J. Intelligent Syst. Appl. Eng.*, vol. 12, no. 4, pp. 1323–1346, Jul. 2024. [Online].